
Linux Kernel Auditing Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *November 7th, 2019*

Introduction

1. Kernel bugs are the highest-value class: a kernel primitive defeats every user-space mitigation boundary at once (sandbox, MAC, process isolation). Correspondingly the audits are harder: bigger codebase, more concurrency, guard pages replaced by trust.
2. The module covers kernel attack surface enumeration, driver auditing discipline, kernel fuzzing, and the exploitation model differences from user land. Assume the source auditing lecture's bug patterns as prerequisites — they all recur in the kernel with different primitives.

Kernel Attack Surface

1. Generic kernel attack surface (what any local attacker can reach):
 - (a) Sockets and netlink sockets: protocol handlers (AF_INET families, netlink message parsers) are classic remote-to-kernel and local-to-kernel surfaces respectively. Netlink family handlers historically have weak length validation.
 - (b) UNIX domain sockets: SCM_RIGHTS fd passing and the message parsers.
 - (c) Crash handling facilities: coredump pipelines, ptrace, prctl handlers. The debuggerd/crash_dump model from android_questions.txt is Android's version of this surface.
 - (d) Kernel helpers exposed for user convenience: keyctl, bpf, io_uring, perf events, userfaultfd. Each is a parser + state machine reachable by unprivileged callers where available.
 - (e) Filesystems: both the VFS switch layer and individual filesystem parsers (on-disk format validators). mountable-media parsers process attacker-authored bytes with kernel privileges and historically yield everything from OOB reads to arbitrary writes; CVE-2016-5195 (Dirty COW) was reachable through a private mapping race code path.
 - (f) Race conditions: kernel preemption points inside syscall families create windows; a class of their own (see Concurrency).
 - (g) GPUs: the largest vendor-written attack surface on modern systems (see Drivers).
2. Android additions map to android_questions.txt items: binder, ashmem, ION, trustzone drivers, adsprpc, /proc and /dev entries of vendor drivers.

3. Method: enumerate the surface the way the source auditing lecture teaches attack surface analysis, but with the kernel-specific reachability question: who can open/reach this (DAC, MAC, capability, namespace), where do untrusted bytes first touch kernel memory?
4. The generic syscall table is the largest single surface, and the reason it is worth reading in slices rather than as a whole. The families that recur in public research:
 - (a) `io_uring` (5.1+): an asynchronous syscall interface with its own object model, its own parser of submission entries, and historically a rich source of lifetime bugs.
 - (b) `eBPF` and the verifier: a bytecode interpreter whose verifier is the security boundary, and whose own bugs are a distinct research area.
 - (c) `keyctl`, `userfaultfd`, `perf_event_open`, `futex`, `signalfd`, `pidfd`, `memfd`, and `splice/tee`: small interfaces that each carry state, lifetime, and race bugs.
 - (d) `AF_ALG` (kernel crypto API), `AF_PACKET`, `AF_NETLINK`, `AF_VSOCK`, `AF_TIPC`, `AF_XDP`: network protocol handlers, where the parser is the attack surface.
 - (e) Filesystem and mountable-media parsers: a mounted image is attacker-authored bytes processed with kernel privilege (the classic CVE source).
 - (f) USB and Bluetooth stacks: reachable by plugging in a device or by radio, no local access required.
5. The generic vs. vendor split from the source auditing lecture applies exactly here: the mainline syscall surface is fuzzed by `syzkaller` and audited by maintainers for years; the vendor driver and the vendor patch on top of mainline are neither. The highest-yield enumeration is a diff of a vendor kernel against the mainline version it was based on.

Driver Auditing Section

1. DAC and MAC, and how each is implemented. DAC is the classic permission bits checked at the VFS (`inode_permission` → `generic_permission`, with POSIX ACLs and the sticky bit as modifiers). MAC (SELinux, SMACK, AppArmor, and the newer BPF-LSM) adds policy hooks at the syscall gateways and other entry points through the LSM framework (`include/linux/lsm_hooks.h`, called from `security/security.c`). Neither is a bug boundary: a driver reachable by a sandboxed-but-labeled process is reachable, period. The security models module is the full treatment of this paragraph.
2. Identify what is available `/dev/` and `/proc/` and to whom: walk the device tree on the target, check DAC modes and SELinux labels (`ls -Z`), and map which contexts can open which nodes. The vendor-written, least-audited parts exposed to untrusted contexts are the audit queue.
3. Identify fops for each available thing: for each character device the `file_operations` table defines the reachable verbs (`open/read/write/ioctl/mmap/release`). The audit surface of a driver is its fops table plus whatever `sysfs/procfs/debugfs` entries it registers.

4. Focus on things that are implemented by vendors or third party (lower hanging fruit): GPU (Mali, Adreno/kgsl), modem, sensor, and ION-adjacent code. Mainline-audited code is fuzzed for years; the vendor driver written by the contracted vendor is not.
5. Make sure you understand what open and release are doing and are matched up: leaked references, reference counts incremented but never decremented, per-fd state stored in globals — all give UAF/overflow primitives.
6. Focus on mmap and that they check parameters correctly: kernel mmap handlers take (vma); the checks are on `vma->vm_pgoff`, `vm_end - vm_start` against the device memory region and `remap_pfn_range` arguments. The classic bug: offset+size computed in u32 (wraparound), or remapping more than the device region — giving user pages of kernel physical memory. The `model_driver.c` lab companion reproduces the wraparound shape.
7. Focus on extensible drivers such as ION: allocator interfaces taking user-side size/flags; each new handle type is a new parser.
8. Focus on things in the driver which are overrode in higher level like /proc level: overrides whose checks disagree with their fops base implementations (per-file .read vs driver's internal handler) are check-disagreement candidates.
9. The mechanical checklist for each driver verb, because the bugs are the same shapes the source auditing module names:
 - (a) `ioctl`: is the command number validated (`_IOC_TYPE/_IOC_NR/_IOC_SIZE`), or is the whole unsigned long arg used as an index, a size, or a pointer? Is there a `compat_ioctl` for 32-bit callers, and does it validate the same fields? A 64-bit struct copied into a 32-bit layout is the classic mismatch.
 - (b) **Argument copy**: every `copy_from_user` return value checked; every struct fully initialized before use (a partially filled struct is the uninitialized-memory pattern); every size derived from user input bounded against the destination.
 - (c) `mmap`: `vma->vm_pgoff`, `vm_end - vm_start`, and the device's own region validated against each other, and `remap_pfn_range/vm_insert_page` arguments derived from validated values only.
 - (d) `poll/read/write`: wait-queue lifetime, the buffer copied out on read (an uninitialized or over-long copy-out is an info leak), and the file-private state's lifetime across release.
 - (e) **Reference counting and lifetime**: each open paired with a release, each get paired with a put, and each object reachable from a file held by a reference. An unbalanced pair is the refcount-borrow pattern at kernel scale.
 - (f) **The higher-level override**: a `procfs/sysfs` write handler that re-implements a permission check differently from the fops it shadows, so the two checks disagree.
10. The audit-queue rule applied to drivers: mainline drivers are fuzzed by syzkaller and reviewed by subsystem maintainers for years; the vendor drop of the same

driver is neither. Diff the vendor driver against the mainline version to see exactly what was added, and read the added code first.

11. Practical enumeration of the target's driver surface, in order: `ls -l /dev` for the character devices, `cat /proc/devices` for the registered major numbers, `ls /sys/class/*/` for the class devices, `cat /proc/kallsyms` (if readable) for the driver symbols, and `modinfo` for the module parameters. Then read the mode bits and the SELinux label of each node to know who can reach it.

Kernel Fuzzing

1. kAFL: hardware-assisted (Intel PT) feedback fuzzing for kernel and hypervisor targets; works without source instrumentation, useful for closed kernel modules.
2. trinity: syscall argument randomizer, older, less feedback; still useful for quick sweeps.
3. syzkaller: descriptor-based syscall generation with coverage feedback (KCOV); the production standard for Linux kernel syscall fuzzing. Descriptor language encodes kernel structs so the fuzzer learns which argument shapes drive deep paths.
4. razer: research system targeting data races in kernel drivers (fine-grained race-window scaling).
5. Kernel fuzzing harness specifics: cover-feed costs (KCOV overhead), recovery of kernel state between inputs (VM snapshots or reboot loops), and the sanitizer arm: KASan (UAF/OOB), KCSan, KMSan equivalents.
6. syzkaller in the detail that matters: descriptions are programs in the `syzlang` language, each syscall with its argument types; `syz-manager` runs a pool of VMs, each with `syz-executor`; `syz-repro` and `syz-crash` minimize and deduplicate; and coverage comes from KCOV. Because the descriptor encodes kernel structs, the fuzzer's generated arguments are already plausible, which is the generative half of the fuzzing module applied to syscalls.
7. The kernel fuzzing loop is more expensive than a userspace one for three reasons: every input needs a fresh kernel state (VM snapshot or reboot), a crash takes the whole VM down, and the feedback channel (KCOV) is slower than a shared-memory bitmap. That cost is why kernel fuzzing is measured in cores $\times$ time rather than in executions per second.
8. Where the bugs are, given a fixed budget: the syscall families above, the vendor drivers, and the filesystem parsers. `syzbot` and the public kernel CTF programs are the practical proof that this works; a report from either is a starting point, not a finding.

Android Specific Attack Surfaces

1. Kernel side: `/proc` and `/dev` entries (gpu, ashmem, binder, vendor drivers) as enumerated in the Driver section; plus Android-specific kernel additions (binder.c, ashmem, ION) that upstream does not share the same audit depth on.
2. User side: all the services and how to go after them — `android_questions.txt` covers service enumeration, binder transactions, and the debuggerd pipeline; treat the user-facing service exposure as a client to the kernel surfaces above, not as a separate target class.
3. The Android kernel is a vendor fork of a specific mainline release, and the fork is the audit queue: the Android Common Kernel (ACK) plus the vendor's own patches plus the vendor modules. Diffing the device's kernel source against the matching ACK branch is the fastest enumeration of the added surface.
4. Android hardening that changes the reachability of the surfaces above, and that therefore must be read first: SELinux per-domain policy with `neverallow` constraints, per-app `seccomp` filters, the app sandbox's per-UID isolation, and the Generic Kernel Image (GKI) boundary that keeps vendor modules in a separate interface.
5. Android debugging builds are the practical entry point: a `userdebug` or `eng` build has `adb root`, a permissive-or-disableable SELinux, and often KASAN enabled, which is the state in which the kernel fuzzing loop above is actually run.

Concurrency Inside the Kernel

1. Every syscall can run preemption-point-concurrent with itself on another CPU: every global or shared structure reachable from a syscall is racy unless an explicit lock says otherwise.
2. Audit discipline: for each fops verb, list the shared state touched (module globals, driver context, file-private), and the lock that protects it (or an explicit "none"). Any pair of verbs sharing mutable state without a common lock is a candidate race.
3. `get_user_pages` and `mmap`-expansion races, table-realloc races (handle tables growing while another thread indexes), and driver reference-count/check pairs are the recurring shapes — the source auditing lecture's `refcount-borrow` and `path-race` sections transfer directly.
4. The kernel locking vocabulary to name in the audit, because "no lock" is the finding:
 - (a) `spinlock_t` (atomic, no sleeping) and `mutex` (sleepable) protect a shared structure; the audit question is whether *every* access to that structure holds it.
 - (b) `rcu_read_lock` is a reader-side convention, not a lock: it protects only while the reader holds a reference, and `rcu_dereference/rcu_assign_pointer` are the rules that make the convention real.

- (c) `seqlock_t` and `rwlock_t` are the optimistic and reader/writer variants; a `seqlock_t`'s retry loop is a place where a check can be re-run on a changed value.
 - (d) `atomic_t`/`refcount_t`/`kref` are the lifetime primitives; an `atomic_dec_and_test` that frees is the kernel form of the source auditing lecture's `refcount` bug.
 - (e) `waitqueue`, `completion`, `workqueue`, `timer`, `tasklet`, and `softirq` contexts are where a callback runs later than the check; anything the callback reads may have changed.
5. The tools: `lockdep` proves the locking discipline at runtime, `KCSan` finds the data races, and `CONFIG_DEBUG_*` plus `syzbot`'s race reports are the public evidence. In an audit, a function that takes a lock in one path and not another is a candidate; `lockdep` is the check.
 6. The audit question for every fops verb, restated as a table: shared state touched, lock that protects it, and the lifetime of any pointer held across a sleep. Any pair of verbs sharing mutable state without a common lock is a candidate race; any pointer held across a sleep without a reference is a candidate UAF.

Kernel Exploitation

1. Differences between physical mem, io mem, `vmalloc` and `kalloc` memory: `kmallo` (slab/slub: physically contiguous, small sizes, active freelists), `vmalloc` (virtually contiguous, page-mapped, larger allocations), `ioremap`/device memory (MMIO regions, side effects on access), physical pages obtained by page allocator. Exploit primitives differ per region: `kmallo` corruption operates on freelist metadata (SLUB freelist pointer hardening, `CONFIG_SLAB_FREELIST_HARDENED`), `vmalloc`/page-level attacks target page tables.
2. Kernel heap layouts and spraying: the heap is shared across all processes and kernel activity — the attacker's own syscalls (pipe buffers, `msg_msg`, `setxattr`, `userfaultfd`-staged frees) double as spray primitives. Cross-cache and cross-ring attacks (user-allocated pages reinterpreted as kernel objects on type confusion) are the modern framing.
3. Spraying techniques: many small same-size allocations (`msgget`, `ftruncate` on pipe, `add_key`) to groom slab pages; heap-feng-shui (allocate/free to carve holes) mirrors user-land grooming from the source auditing heap-qualities list with different size-class arithmetic.
4. Post-corruption goal differs: kernel land prizes elevation (`cred` struct overwrite, `modprobe_path` overwrite, `sid` loop through process credentials) and the read-modify-write target set is the kernel's own metadata, so an in-place arbitrary write already wins if you pick the right target.
5. The user-to-kernel boundary is a small mitigation map of its own, and each row changes what the chain needs:
 - (a) **SMEP/SMAP**: the kernel cannot execute or read user pages, so `ret2usr` and a user-mapped fake stack both fail; the chain must stay in kernel memory (kernel ROP, a pivot into a kernel object).

- (b) **KPTI**: user page tables are separate, so a kernel address is not mapped in userspace; the leak and the chain must use kernel addresses, and `ret2dir` (pivoting through the direct map) becomes the workaround for SMAP.
 - (c) **KASLR**: the kernel base is randomized per boot, so the chain needs a kernel leak (`kallsyms`, a `seq_operations` read, a `msg_msg` over-read) before it can name a gadget or a function.
 - (d) **Stack canary** (`CONFIG_STACKPROTECTOR`): an overwrite past a stack buffer must reproduce the per-task canary, which is why the canary is usually leaked rather than brute-forced.
 - (e) **SLUB freelist hardening** (`CONFIG_SLAB_FREELIST_HARDENED/_RANDOM`): the freelist pointer is mangled and the next free object is randomized, so the classic “overwrite the next pointer” groom is no longer deterministic.
 - (f) **usercopy hardening**, `INIT_ON_ALLOC/INIT_ON_FREE`, and `STATIC_USERMODEHELPER`: remove the cheap uninitialized-read, the reused-object, and the `modprobe_path` primitives respectively. Read the target’s config before choosing a technique.
6. The techniques that recur in public research, and the bug each needs: a UAF or double-free groomed into a sprayed object (`msg_msg`, `pipe_buffer`, `sk_buff`, `tty_struct`, `seq_operations`, `setxattr`); an overflow into a length field (`msg_msg`’s `m_ts`) for an out-of-bounds read or write; an in-place write of `cred` for a direct elevation; and `userfaultfd` or `FUSE` to widen a race window (the debugging module’s timing problem at kernel scale).
 7. The elevation goal is usually one of a small set: overwrite the current task’s `cred` (or `commit_creds(prepare_kernel_cred(0))` if control flow is available); overwrite `modprobe_path` or `core_pattern` for a controlled helper execution; overwrite a function pointer used by a subsystem running in a privileged context; or corrupt the page tables directly.
 8. Dirty COW (CVE-2016-5195), Dirty Pipe (CVE-2022-0847), and DirtyCred are the class of bugs whose whole exploit is an in-place write into a read-only object; they are the reason the read-modify-write target set above is the one that matters, not a control-flow hijack.
 9. Cross-cache attacks (reclaiming a freed slab page for a different cache) are the modern generalization of the userspace grooming technique, and they are why the allocator’s `CONFIG_*` hardening must be read before the primitive is priced.

Conclusion

The kernel is more source auditing than anything else: the bug classes do not change; they scale (concurrency everywhere), the guard rails change (nobody isolates you from the kernel), and the payoff structures differ (elevation primitives rather than shell-code). Combine: surface enumeration from source auditing, the fuzzing module’s `syzkaller/KASan` stack, and the reversing module’s toolset for vendor blobs.

The module in one sentence: enumerate who can reach which kernel memory (DAC, MAC, capability, namespace), read the vendor half of the surface first, and price every finding against the kernel’s own mitigation map (SMEP/SMAP, KPTI, KASLR,

canary, SLUB hardening, usercopy hardening). The kernel is the one target where the mitigation map and the attack surface are both larger than any userspace target, and where the audit-queue rule pays the most.

References

1. syzkaller: github.com/google/syzkaller
2. kAFL: github.com/RUB-SysSec/kAFL
3. KCOV documentation: [kernel.org doc \(kcov.rst\)](https://kernel.org/doc/kcov.rst)
4. LWN kernel development articles (lwn.net/Kernel/Index)
5. Dirty COW writeups (dirtycow.ninja), Dirty Pipe (CVE-2022-0847)
6. `android_questions.txt` in this repo (interview-grade kernel/ Android surface notes)
7. Model of Source Trees: [kernel newbies \(kernelnewbies.org\)](https://kernelnewbies.org)
8. Kernel Self Protection Project (kspp) hardening recommendations and the Linux kernel documentation on LSM, KASAN and seccomp
9. kernelCTF and syzbot public reports (google.github.io/security-research, syzbot.appspot.com)